

Generic MinimumLinuxBoot for RTL Simulations

Google Summer of Code 2025 Proposal-The FOSSi Foundation

April 8, 2025

1 Basic Information

- **Name:** Kislay Arya
- **Email:** kislayarya536@gmail.com
- **GitHub Handle:** kislay536
- **LinkedIn Profile:** Kislay Arya
- **Nationality:** Indian
- **University:** Indian Institute of Technology Kharagpur
- **Degree:** B. Tech (2022-2026)
- **Contact:** (+91)7250067420
- **Timezone:** Indian Standard Time (UTC +5:30)

2 About Me

I am a third-year undergraduate student at Indian institute of Technology Kharagpur, pursuing a B.Tech in Electronics and Electrical Communication Engineering with a minor in Computer Science. I am passionate about computer architecture, hardware security and hardware design in general. Over the past year, I have worked on projects like security analysis of modern server grade processors, designing tools to aid the verification process of hardware designs and designing secure systems to mitigate cache based side channel attacks.

Open-source has been instrumental in my learning journey, providing access to real-world systems, simulators, and community-driven innovation. I see GSoC as a unique opportunity to work with experienced mentors and contribute to impactful tools that empower researchers and engineers globally. I want to be a part of GSoC not only to grow technically but also to give back to the open-source communities that have shaped my learning. By contributing to the open-source hardware ecosystem through this project, I aim to create tooling that reduces entry barriers for researchers and students like me, who are eager to explore systems and computer architecture but are limited by free tools available. I believe the work done here will have a lasting impact and further democratize access to advanced simulation environments.

3 Why This Project?

Simulations are essential in hardware design, enabling architects and engineers to validate and optimize system behavior before committing to costly physical prototypes. Detailed RTL simulations, such as those provided by OpenPiton-Verilator, offer cycle-accurate insights that are critical for verifying low-level hardware interactions and timing. However, their high fidelity comes at the expense of simulation speed—booting a full Linux system in RTL can take several hours, making rapid prototyping and debugging impractical. To overcome this bottleneck, faster simulation methods using high-level emulators

like SPIKE and QEMU are often employed. These platforms allow for quick booting of Linux and rapid testing of software, but they lack the cycle-accurate detail needed to catch subtle hardware design issues.

This project introduces an innovative checkpoint-restore framework that combines the speed of high-level emulation with the accuracy of RTL simulation. By doing so, this project serves several important purposes:

- **Accelerated Verification:** It significantly reduces the time needed to reach a working Linux state in RTL simulations, thereby expediting hardware validation cycles.
- **Enhanced Productivity:** Developers and researchers can bypass lengthy boot sequences, allowing more time to focus on optimizing and debugging critical hardware features.
- **Bridging Simulation Domains:** It harmonizes the trade-off between speed and accuracy, providing a tool that delivers both rapid prototyping and detailed verification.

4 Proposal Description

4.1 Overview

This project aims to enable seamless transition between software-based CPU simulation (such as QEMU or SPIKE) and RTL-based simulation (using OpenPiton with Verilator). The main goal is to boot Linux in a high-level simulator, capture the essential system state, and restore it in an RTL simulation to continue execution. This approach allows for efficient and flexible hardware/software co-simulation, crucial for architectural validation and security research.

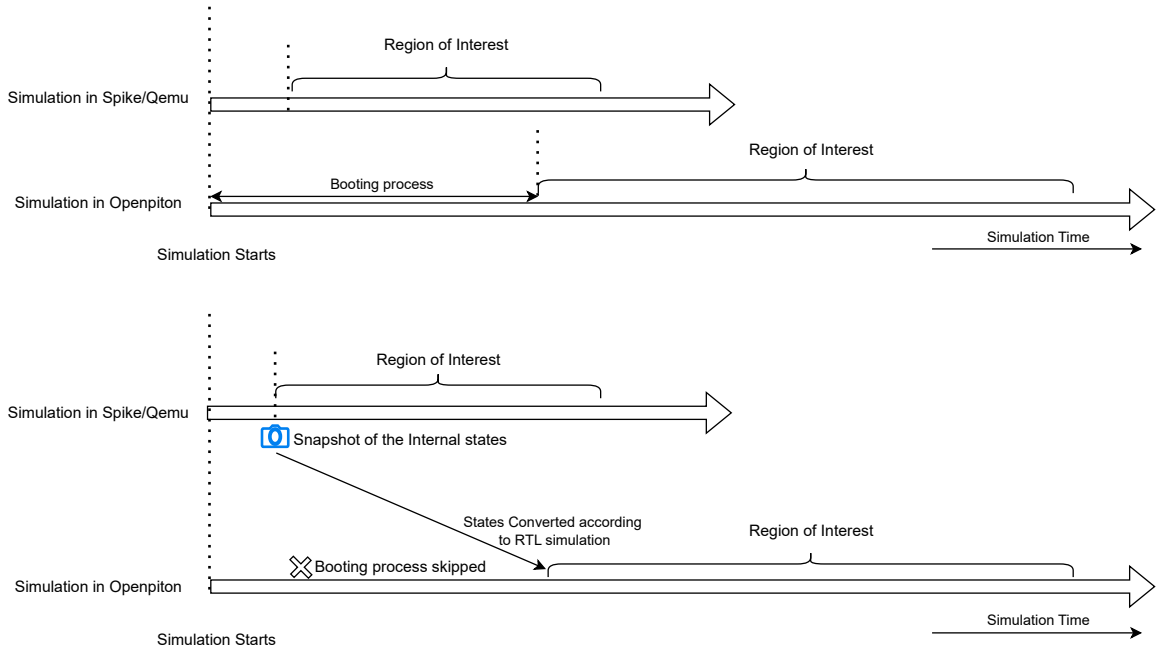


Figure 1: Overview

4.2 Objectives

1. Task 1- Identify Minimal State for Checkpointing

Determine the essential system state needed to resume execution. This includes memory contents, CPU register states, and critical MMU/TLB configurations.

2. Task 2- Checkpointing in QEMU/SPIKE

Develop a method to extract and store the identified system state during Linux boot.

3. Task 3- State Restoration in OpenPiton

Implement a mechanism to load the checkpointed state into OpenPiton’s Verilator-based simulation, ensuring a seamless transition from QEMU/SPIKE.

4. Task 4- Integration with OpenPiton’s Simulation Infrastructure

Modify OpenPiton’s memory and execution state management to support checkpoint-based booting.

5. Task 5- Validation and Testing

Test the entire flow by running Linux applications after restoring the system state in OpenPiton, ensuring correct execution.

4.3 Expected Outcome

By the end of this project, we aim to achieve a functional pipeline where Linux boots in QEMU or SPIKE, checkpoints its state, and resumes execution in OpenPiton’s Verilator simulation. This will provide a powerful tool for architecture research, enabling detailed performance analysis and security validation within a realistic execution environment.

4.4 Leveraging QEMU’s Migration Framework for Checkpointing

In my project, I plan to utilize **QEMU’s migration framework** to implement an effective checkpointing mechanism. This framework, originally designed for live migration of virtual machines (VMs), provides robust infrastructure for saving and restoring the complete state of a VM—including CPU registers, device states, memory contents, and more. This capability is essential for enabling a transition from QEMU or SPIKE simulations to **OpenPiton RTL simulation**, where a consistent and complete snapshot of the system state must be transferred.

4.4.1 Understanding VMState in QEMU

The core concept in this framework is the `VMStateDescription` structure. This structure is used to define how the internal state of a QEMU device or component is saved and restored. Each device participating in the migration process must declare its state using this structure.

Key Fields in VMStateDescription:

- **Name:** Identifier for the device/component.
- **Version ID:** Indicates the version of the state format, useful for maintaining compatibility.
- **Fields:** An array of state elements (e.g., integers, structures) to be saved/restored.

4.4.2 How VMState Works

When migration or checkpointing is initiated, QEMU traverses the list of registered `VMStateDescription` objects and serializes the listed fields to an output stream (such as a file or network). During restoration, it deserializes this data to recreate the original state.

This deterministic snapshot of state ensures a consistent transfer across different simulation environments, enabling the same execution state to be continued from the checkpoint.

4.4.3 Example: Defining a VMStateDescription

Here is a simplified example of defining a `VMStateDescription` for a hypothetical device:

```
static const VMStateDescription vmstate_my_device = {
    .name = "my_device",
    .version_id = 1,
    .minimum_version_id = 1,
    .fields = (VMStateField[]) {
        VMSTATE_UINT32(some_register, MyDeviceState),
    }
};
```

```

        VMSTATE_BOOL(enabled, MyDeviceState),
        VMSTATE_END_OF_LIST()
    }
};

```

In this example:

- `some_register` is a 32-bit register.
- `enabled` is a boolean flag.

These values will be automatically saved and restored during migration or checkpointing.

4.4.4 Implementing Checkpointing with VMState

For this project, I will:

- **Define VMStateDescriptions:** For each relevant QEMU component (CPU, memory controllers, etc.), I will identify the internal states that must be saved and define corresponding VMState structures.
- **Integrate Save/Load:** I will invoke QEMU’s built-in migration routines to save the full system state to a file and restore it later during RTL simulation.
- **Ensure Compatibility:** By maintaining versioning, I will ensure the checkpoint format remains compatible across different versions of the simulator.

4.4.5 Modifying Migration Data Structures

As new features or architectural changes are introduced:

- **Use Subsections:** For optional or extended state, I will define subsections inside VMStateDescription. This ensures old formats can ignore unknown data.
- **Manage Versioning:** Any changes in structure will be reflected via `version_id` updates, allowing backward/forward compatibility.
- **Rigorous Testing:** Every change will be validated via checkpoint-restore cycles across QEMU and OpenPiton RTL to ensure reliability.

This framework will allow me to efficiently checkpoint simulation states from QEMU and later resume execution in RTL simulations, ensuring high fidelity across both platforms.

For more details, refer to the official QEMU migration documentation: <https://www.qemu.org/docs/master/devel/migration/main.html>

4.5 Checkpointing with Spike

Spike, the RISC-V ISA simulator, supports saving and restoring architectural checkpoints using a wrapper script that leverages the simulator’s `--dump-dts`, `--dump-device-tree`, and `--dumpstate` options. The file `generate-ckpt.sh`¹ provided in the Chipyard repository automates this process. Below is a breakdown of how the checkpointing is achieved using Spike.

Script Overview

The script performs the following steps:

1. Builds the Linux image using `make` from the Chipyard directory.
2. Launches Spike with the appropriate options to run the target Linux binary.
3. Enables checkpointing at a specified instruction count using the `+checkpoint-restore` and `+checkpoint-dir` options.

¹<https://github.com/ucb-bar/chipyard/blob/master/scripts/vcs-sim/generate-ckpt.sh>

4. Dumps the state to a directory using `--dumpstate`, which includes:

- `memory.vmem` – memory snapshot.
- `state.json` – processor state.
- `device_tree.dts` – flattened device tree source.

4.5.1 Checkpointing with Spike: Key Commands and Usage

Spike, being the reference RISC-V ISA simulator, offers robust support for checkpointing architectural state. This is essential for ensuring deterministic and reproducible simulation workflows. The `generate-ckpt.sh` script from the Chipyard framework leverages Spike’s checkpointing capabilities to capture both memory and processor state. The following command is central to this mechanism:

```
$SPIKE_PATH \
--isa=rv64gc \
--priv=msu \
-p2 \
--dump-dts \
--dump-device-tree device.dts \
--dumpstate \
-m0x80000000:0x20000000 \
-H \
$PK_PATH \
$BINARY \
+checkpoint-restore=0 \
+checkpoint-dir=$CHECKPOINT_DIR
```

Each flag plays a specific role in orchestrating a complete architectural state dump:

- `--isa=rv64gc`: Specifies the instruction set variant (64-bit RISC-V with general extensions).
- `--priv=msu`: Configures the privilege levels to support machine, supervisor, and user modes.
- `--dump-dts`, `--dump-device-tree`: Instruct Spike to emit the flattened device tree (FDT) which encodes memory map and device configuration. This is critical for reproducing the system environment during restore.
- `--dumpstate`: Triggers a full dump of the architectural processor state (registers, control state, etc.) and memory.
- `-m0x80000000:0x20000000`: Defines the memory layout for the simulation (e.g., 512MB starting at physical address `0x80000000`).
- `+checkpoint-restore=0`: Puts Spike into checkpoint-saving mode (as opposed to restoring from a previous checkpoint).
- `+checkpoint-dir=$CHECKPOINT_DIR`: Sets the target directory where the complete architectural checkpoint will be saved.

Checkpoint Generation

The script simplifies the checkpointing workflow. To save a checkpoint from a Linux binary, one needs only to run:

```
./generate-ckpt.sh <path-to-linux-binary> <checkpoint-dir>
```

This command initiates Spike with the appropriate parameters to generate a full architectural snapshot, including memory content, register files, and device tree. The generated checkpoint serves as a frozen system state that can be restored later either within Spike or passed to another simulator like an RTL implementation.

Checkpoint Restoration

To restore from a previously generated checkpoint, Spike is launched with modified flags:

```
+checkpoint-restore=1 \  
+checkpoint-dir=<checkpoint-dir>
```

This instructs Spike to initialize its architectural state from the given directory, bypassing the need to re-execute the entire boot and initialization sequence. It ensures deterministic continuation of execution from the saved state.

Relevance to Our Work

The ease of saving and restoring complete system state makes Spike an invaluable frontend for our proposed simulation ecosystem. The state captured by Spike can be translated into an intermediate format (e.g., gem5-compatible checkpoints), enabling cross-simulator execution and hardware-software co-validation. In particular, this serves as the initial architectural state when porting into lower-level RTL platforms like FireSim or OpenPiton, as it bypasses the need to simulate the boot process repeatedly. By preserving memory layout, register content, and system configuration, the checkpoints support reproducible research, fast design-space exploration, and verification of architectural modifications across simulator boundaries.

4.6 Proposed Work

The goal of my project is to enable a seamless and automated transition from functional simulations using **QEMU** and **Spike** to **RTL-level simulation** in **OpenPiton**, by establishing a robust **checkpointing mechanism**. This involves capturing the architectural state of the system after a minimal Linux boot and restoring it in the RTL simulation environment for further analysis or verification.

To achieve this, I plan to leverage existing infrastructures provided by QEMU and Spike for saving system state.

4.6.1 QEMU-Based Checkpointing Using Migration Framework

QEMU provides a built-in *migration framework* that supports serializing and saving the state of a virtual machine. Instead of using this mechanism for live migration between hosts, I will repurpose it for **checkpointing**—i.e., saving processor, memory, and device state to disk so that simulation can be resumed elsewhere.

I will:

- Extend or reuse QEMU’s `VMStateDescription` structures to describe the relevant architectural state (e.g., registers, PC, memory layout).
- Create checkpoint files at specific simulation points (e.g., after Linux boot completion).
- Implement logic to extract and reformat this checkpoint information for compatibility with OpenPiton RTL simulation.

This provides a structured and reliable way to extract simulation state directly from QEMU, enabling deterministic and restartable RTL simulations from realistic software checkpoints.

4.6.2 Spike-Based Checkpointing via Chipyard Scripts

In parallel, I will use **Spike**, the official RISC-V ISA simulator, which supports architectural checkpointing through the **Chipyard** infrastructure. Chipyard provides a script—`generate-ckpt.sh`—that can generate snapshots of the machine state at specified cycle counts or PC addresses.

I will:

- Use or adapt this script to generate checkpoints after minimal Linux boot (e.g., after init or shell prompt).
- Validate that checkpoints contain all relevant state: PC, general-purpose registers, privilege levels, page table pointers, and memory contents.
- Write tooling to translate these checkpoints into a format ingestible by OpenPiton RTL testbenches.

This approach is advantageous due to Spike’s compatibility with RISC-V ISA and deterministic behavior, ensuring that checkpoints are clean and architecture-compliant.

4.6.3 RTL Integration and Execution

Once checkpoint data is obtained from QEMU and/or Spike, the final stage of the project will focus on **injecting this data into the RTL simulation** (OpenPiton):

- Develop Verilog/C/Python interfaces for loading register file contents, memory images, and PC into the RTL simulation.
- Verify that execution continues from the correct state, and Linux resumes or executes post-checkpoint payloads (e.g., running user programs or kernel functions).
- Automate the full pipeline from high-level simulation → checkpoint generation → RTL resume.

This proposed workflow eliminates the need to boot Linux entirely within the RTL simulation, saving extensive time while preserving realism. It also builds on well-established simulator frameworks (QEMU and Spike), enabling reproducibility and easier debugging.

4.7 Execution Plan

The following phased execution plan outlines how I intend to implement the proposed checkpoint-resume flow using QEMU and Spike:

- **Phase 1: Setup and Minimal Linux Boot on QEMU and Spike**
 - Configure a minimal Linux image that boots successfully on both QEMU and Spike.
 - Automate boot logs parsing to detect successful boot events (e.g., login prompt).
 - Verify functional correctness of Linux boot in both environments.
- **Phase 2: Implementing Checkpointing in QEMU**
 - Explore QEMU’s migration framework in detail.
 - Identify and define VMStateDescriptions for core architectural elements (register file, PC, memory).
 - Modify QEMU to generate checkpoints using the migration mechanism at runtime.
 - Extract and serialize the relevant checkpoint data into a custom format usable by OpenPiton.
- **Phase 3: Spike Checkpointing Integration**
 - Leverage Chipyard’s `generate-ckpt.sh` script to save architectural checkpoints in Spike.
 - Analyze the file format and structure of Spike-generated checkpoints.
 - Develop transformation logic to convert the Spike checkpoints into RTL-consumable format.
 - Validate fidelity of checkpoints by restarting Spike simulation and ensuring correctness.
- **Phase 4: RTL State Loading Infrastructure**
 - Understand OpenPiton RTL simulation flow and memory/register initialization routines.
 - Develop RTL/C interface modules or harnesses to load checkpointed memory and registers.
 - Validate the pipeline resume by injecting known checkpoint data and verifying execution.
- **Phase 5: Full Pipeline Integration**
 - Automate the full path: QEMU/Spike simulation → checkpoint generation → RTL resume.
 - Create scripts for reproducible and configurable runs across different workloads.
 - Benchmark performance and correctness against full RTL Linux boots.
- **Phase 6: Testing and Documentation**
 - Perform extensive testing across multiple checkpoints and edge cases (e.g., user programs, syscalls).
 - Document each module, script, and methodology.
 - Prepare reproducibility guide and validate by repeating pipeline on different systems.

- Resume simulation in gem5 to validate correctness and optionally re-checkpoint.
- Translate gem5 checkpoints into a format suitable for RTL simulators like OpenPiton.

gem5’s checkpointing infrastructure is well-documented and provides the ability to serialize CPU, memory, and system state (gem5 Checkpoints Documentation). This makes it suitable not just for simulation continuation but also as a standardized interchange format across simulation tools.

Analogous to LLVM Intermediate Representation (IR)

This proposed use of gem5 as an intermediate state format is analogous to the way LLVM IR operates in compiler toolchains. Just as LLVM IR abstracts various source languages into a common intermediate form to enable cross-platform compilation and optimization, gem5 checkpoints can serve as a common abstraction for simulation state, enabling interoperability between functional, timing, and RTL simulators. **Benefits of Using gem5 as IR**

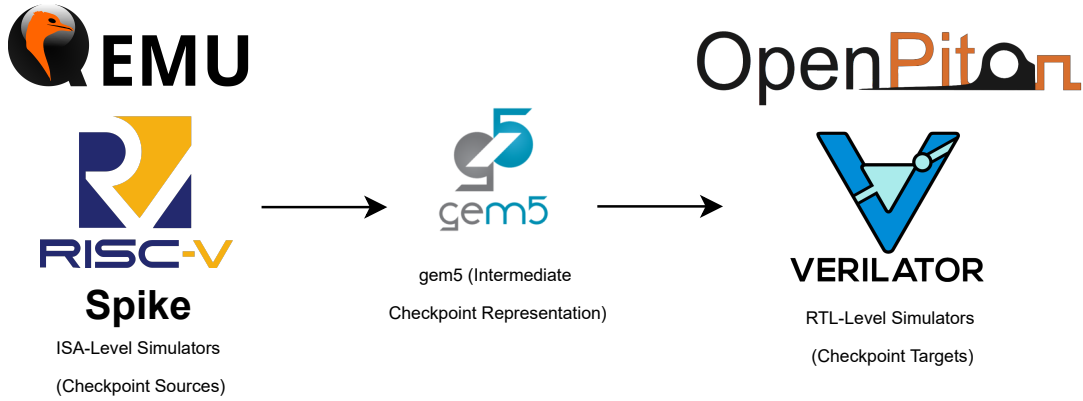


Figure 4: Extended Checkpoint Translation Framework

- Facilitates easier switching between simulators of varying complexity and performance.
- Establishes a common ground for research efforts focused on software-hardware co-design.
- Provides a standardized and reusable checkpoint format for future extensions.
- Enables fine-grained architectural state inspection, modification, and instrumentation.

Incorporating gem5 as a central hub for simulation state exchange will significantly enhance the flexibility, modularity, and research utility of the checkpointing framework.

5 Project Timeline

5.1 Distribution of Work

The project is structured in alignment with the official GSoC 2025 timeline, focusing on progressively developing a minimal Linux environment on RISC-V platforms, transitioning from software simulators to hardware implementations.

Community Bonding Period (May 8 – June 1) : Lay groundwork for the project, including infrastructure setup and early exploration of gem5/QPoints. Key Tasks:

- Minimal Linux Image Setup: Build and validate minimal Linux images for QEMU/Spike. Document the boot process and automate success detection (e.g., parsing login prompts).
- OpenPiton RTL Familiarization: Study OpenPiton’s simulation flow, memory initialization, and state management.

- **gem5 Integration Prep:** Set up gem5 environment and study its checkpointing infrastructure. Explore QPoints framework for QEMU-to-gem5 state translation.
- **Toolchain Validation:** Ensure compatibility between QEMU/Spike checkpoints and OpenPiton RTL.

Coding Phase I (June 2 – July 13) : Implement checkpointing in QEMU/Spike and begin gem5 integration. Key Tasks:

- **QEMU Checkpointing:** Modify QEMU’s `VMStateDescriptions` to capture CPU registers, memory, and device states. Generate and serialize checkpoints after Linux boot.
- **Spike Checkpointing:** Adapt Chipyard’s `generate-ckpt.sh` to save post-boot checkpoints. Translate Spike checkpoints into OpenPiton-compatible formats.
- **Early gem5 Integration:** Implement QPoints-based translation for QEMU/Spike checkpoints to gem5. Validate gem5’s ability to resume simulations.
- **Initial RTL Integration:** Develop basic Verilog/Python modules to load memory snapshots into OpenPiton.

Mid-Term Evaluation (July 14 – July 18) : Demonstrate functional checkpoints and progress on gem5/RTL integration. Deliverables:

- Working checkpoints from QEMU/Spike.
- Proof-of-concept gem5 checkpoint translation.
- Basic RTL module to load memory/register state.

Coding Phase II (July 19 – August 25) : Finalize full pipeline integration with gem5 and ensure robustness. Key Tasks:

- **gem5 as Intermediate Hub:** Finalize checkpoint translation workflows (QEMU/Spike → gem5 → OpenPiton). Support multi-core checkpoints and dynamic state updates.
- **RTL Execution Pipeline:** Automate end-to-end flow. Validate correctness by running Linux applications post-checkpoint.
- **Performance Benchmarking:** Compare checkpoint-resume time vs. full RTL boot (e.g., 10x speedup target).
- **Edge-Case Testing:** Test syscalls, interrupts, multi-threaded apps, and large memory workloads.

Final Evaluation (August 26 – September 1) : Ensure reproducibility and knowledge transfer. Key Tasks:

- **Documentation:** User guides for checkpoint generation, translation, and RTL integration. API references for gem5/QPoints workflows.
- **Demo Artifacts:** Recorded demo video showing end-to-end pipeline. Publicly shareable test workloads.
- **Community Handover:** Prepare pull requests for OpenPiton/Chipyard repositories. Host a workshop or blog post.

5.2 Week-wise Plan

Period	Objective	Difficulty
Community Bonding (April 30 – May 14)	Study OpenPiton-Verilator simulation flow Explore Linux boot stages and gem5 checkpointing	●
Week 1 (May 15 – May 21)	Identify Linux architectural state (TLB, MMU, regs, etc.) Analyze gem5 checkpoints to extract states(Set A)	●
Week 2 (May 22 – May 28)	Study SPIKE's state manipulation (Set B) Compare with states recorded in gem5 using m5(A and B)	●
Week 3 (May 29 – June 4)	Design minimal checkpoint format Document assumptions and required values	●
Week 4 (June 5 – June 11)	Integrate state restoration into OpenPiton RTL Adapt simulation infra for memory+CPU init	●
Week 5 (June 12 – June 18)	Test booting Linux to userland from minimal state Debug memory/register loading issues	●
Week 6 (June 19 – June 25)	Refine simulation flow Improve reliability of minimal boot	●
Mid-Term (June 26 – June 30)	Present minimal checkpoint-restore pipeline Polish mid-term demo and partial docs	●
Week 7 (July 1 – July 7)	Automate checkpoint extraction scripts Add portability options for toolchains	●
Week 8 (July 8 – July 14)	Optimize pipeline for performance Add test infrastructure and regression cases	●
Week 9 (July 15 – July 21)	Validate corner cases (syscalls, interrupts) Test across kernel configs and core counts	●
Week 10 (July 22 – July 28)	Finalize testing suite Start user documentation with examples	●
Week 11 (July 29 – August 4)	Polish core codebase and cleanup Write developer API guide	●
Week 12 (August 5 – August 11)	Prepare demo video and usage tutorial Package test workloads for release	●
Week 13 (August 12 – August 18)	Handover to community Submit pull requests to OpenPiton repo	●
Final Evaluation (August 19 – Sept 1)	Finalize documentation Conduct knowledge transfer session	●

Table 1: Checkpoint-Restore Minimal Linux Boot: Weekly Breakdown

6 Availability

I will be available full-time during my summer break from May 1 to July 14 and plan to dedicate 30–35 hours per week to the project during this period. Once my academic semester begins, my availability will reduce slightly, and I will continue contributing around 20 hours per week.

To account for the transition period from July 14 to July 22—when I'll be settling into the new semester—I have planned to start working earlier and make substantial progress in the initial weeks. This will help mitigate any impact on the overall timeline. I will also remain in regular contact with my mentors and adjust my schedule if any unforeseen events arise, ensuring steady progress throughout the program.